

Unidad 1

Introducción

Tabla de contenido

1. Introducción.....	3
2. Algoritmo y programa.	3
3. Ciclo de vida de la aplicación.....	5
3.1. Diseño del programa	7
3.1.1. Fase de análisis.....	7
3.1.2. Fase de programación.....	7
3.1.1. Fase de codificación.....	7
3.2. Instalación	7
3.2.1. Edición	7
3.2.2. Compilación	7
3.2.3. Enlazado	7
3.2.4. Pruebas de ejecución	7
3.2.5. Funcionamiento y mantenimiento.....	8
4. Errores.....	8
5. La calidad de los programas	8
6. Documentación de los programas	9

1. Introducción

Un ordenador, básicamente, solo puede realizar 3 tipos de operaciones:

1. Suma, resta, multiplica o divide dos números
2. Compara dos valores y averigua si son iguales, o si el primero es mayor o menor que el segundo.
3. Almacenar y recuperar información.

A través de un programa, se pueden realizar tareas muy complejas combinando esas 3 operaciones.

Las tareas realizadas por un programa pueden ser llevadas a cabo por una persona. Las ventajas de usar una computadora son:

1. La alta velocidad de los cálculos.
2. Total fiabilidad de los cálculos. Siempre y cuando el programa haya sido bien diseñado.
3. Gran capacidad de almacenamiento de información.

2. Algoritmo y programa.



Algoritmo: es un procedimiento paso a paso para resolver un problema en un tiempo finito.

El término proviene del nombre de un matemático persa: Abu Abdallah Mohammad Ibn Musa Al Khwaritzmi (siglo IX).

Una característica esencial de un algoritmo es que **siempre termina**.

Procedimiento: es una secuencia de instrucciones que se pueden realizar de forma mecánica.

Ejemplos de algoritmos:

Juego de 15 monedas.

Este juego de dos jugadores comienza colocando 15 monedas en la mesa.

El jugador uno elige de 1 a 3 monedas. A continuación, el jugador 2 elige de 1 a 3 monedas de las monedas restantes en la mesa. Este proceso se repite hasta que no queden monedas sobre la mesa. El jugador que se ve obligado a tomar la última moneda pierde.

El problema consiste en encontrar una estrategia ganadora que permita al jugador 1 obligar al jugador 2 a tomar la última moneda.

- 1.- El jugador 1 toma 2 monedas
- 2.- El jugador 2 toma x monedas ($x \geq 1$ y $x \leq 3$)
- 3.- El jugador 1 se lleva 4 monedas
- 4.- Repite los pasos 2 y 3 hasta que solo quede 1 moneda.

Cómo arrancar el motor de un coche:

- 1.- Enchufa la llave
- 2.- Pon el equipo en posición neutra.
- 3.- Pisa el acelerador.
- 4.- Gire la llave a la posición de inicio (o al botón de inicio).
- 5.- Si el motor arranca antes de los 6 segundos, deje la llave en posición de encendido.
- 6.- Si el motor no arranca en 6 segundos, espere 10 segundos y repita de 3 a 6 (**pero no más de 5 veces**)
- 7.- Si el coche no arranca, llama al taller.



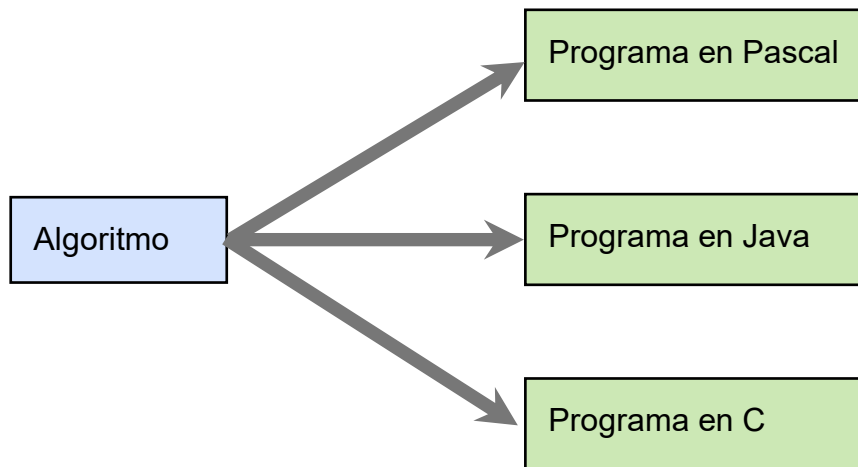
Programa: Es una secuencia de instrucciones que indica las acciones que debe ejecutar un ordenador.



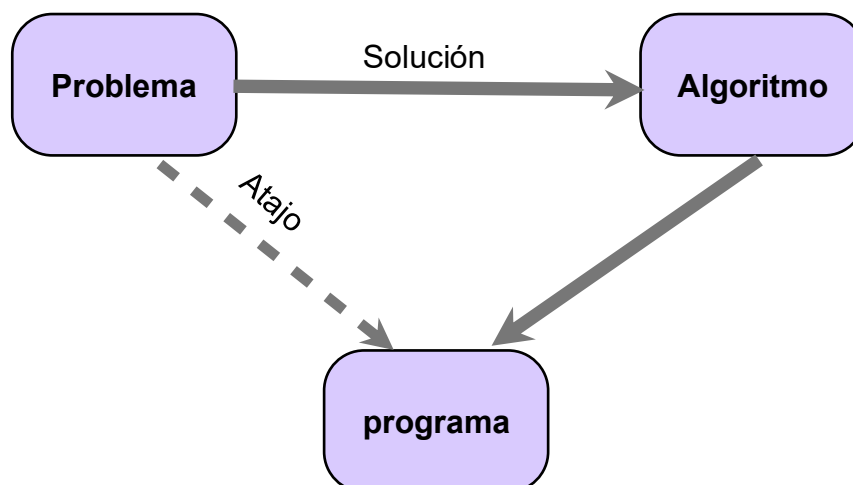
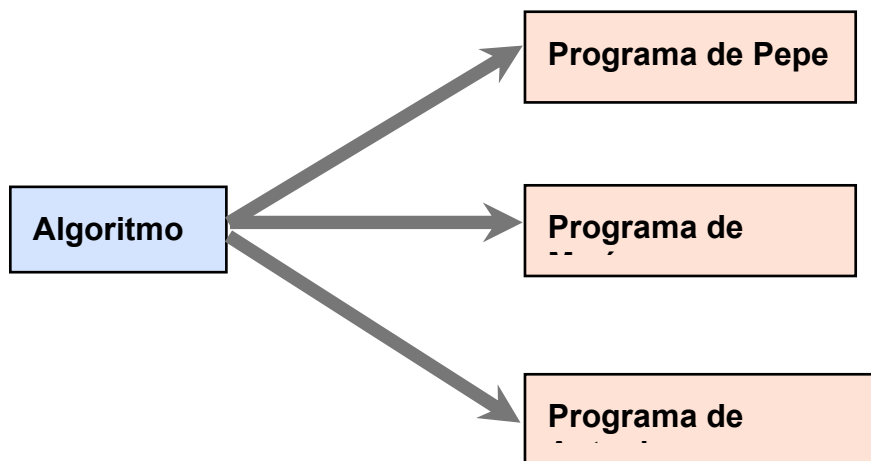
Lenguaje de programación: es un conjunto de reglas, símbolos y palabras especiales que se utilizan para construir un programa.

Lenguajes de programación:

....



Programa en Java

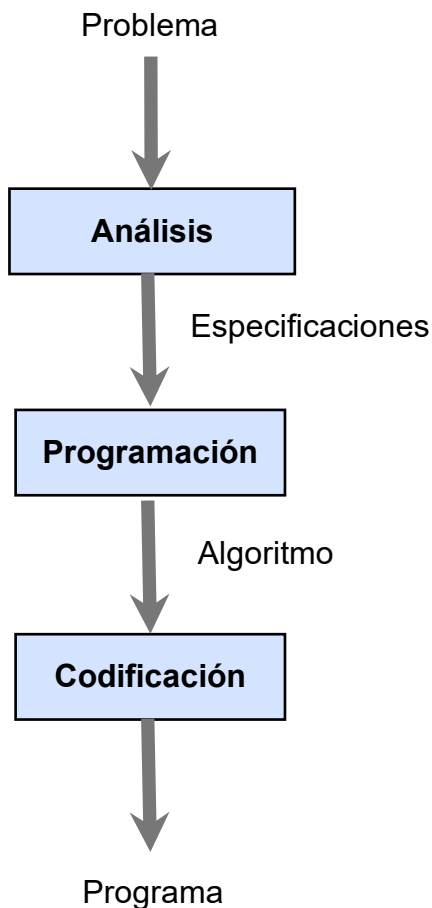


3. Ciclo de vida de la aplicación

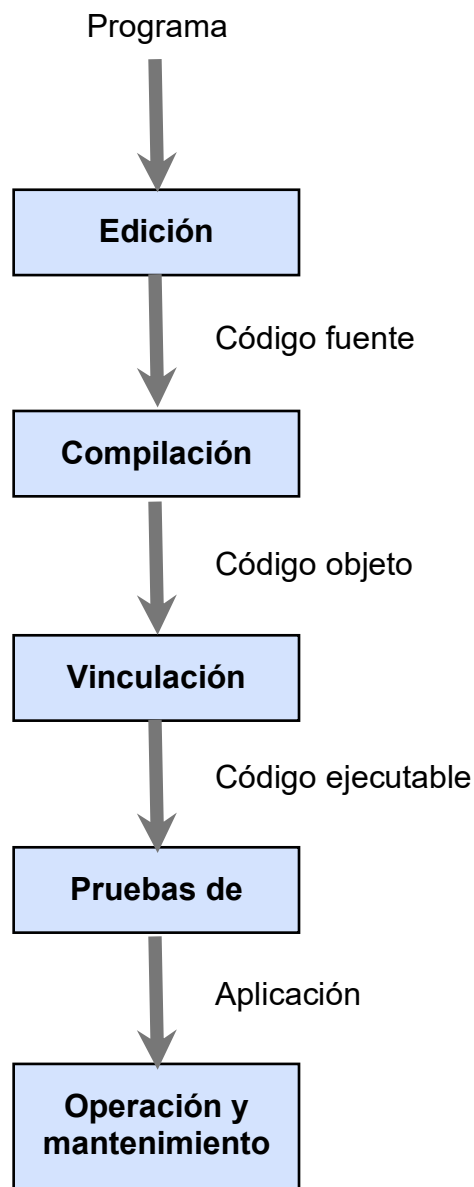
Una **aplicación** está compuesta por uno o varios programas interrelacionados para llevar a cabo una determinada tarea de forma automática utilizando un ordenador.

Ciclo de vida de la aplicación: Es el proceso que se sigue desde la identificación del problema o tarea hasta que tenemos una solución instalada y funcionando en los equipos de los usuarios, siempre y cuando la Aplicación sea útil para ellos.

Fases de diseño:



Fases de instalación:



3.1. Diseño del programa

3.1.1. Fase de análisis

Consiste en el estudio y descripción detallada del problema, incluyendo:

- Análisis de los datos de entrada
- Análisis de los datos de salida
- Relaciones entre la entrada y la salida
- Desglose del problema por módulos.

El resultado de esta fase se conoce como las **especificaciones** del problema, y es un conjunto de documentos que implementan todos estos aspectos.

3.1.2. Fase de programación

Es el diseño de la solución al problema mediante un **algoritmo**. Para representarlo existen diferentes notaciones como diagramas de flujo, pseudocódigo, etc.

3.1.1. Fase de codificación

En esta fase el algoritmo se traduce a un lenguaje de programación específico, como Java. Generalmente es una tarea mecánica.

El resultado de esta fase es el **programa**.

3.2. Instalación

En esta sección se incluye la instalación y configuración del programa en el ordenador.

3.2.1. Edición

Escribir el **código fuente** en el ordenador, guardándolo en un soporte permanente. Esta tarea la lleva a cabo un programa llamado **editor**.

3.2.2. Compilación

Es la traducción del programa en código fuente a lenguaje de máquina. El resultado es el **código objeto**. Esto se hace mediante programas llamados **compiladores e intérpretes**, que además de verificar la corrección de la sintaxis del programa.

3.2.3. Enlazado

Por lo general, es necesario unir algunas piezas de rutinas del sistema u otros módulos (también en código objeto) a nuestro código objeto. Esto lo hace el **enlazador (linker)**. El resultado es un **archivo ejecutable**.

3.2.4. Pruebas de ejecución

Consiste en ejecutar el programa varias veces con diferentes datos de prueba para asegurarse de que funciona como se espera.

3.2.5. Funcionamiento y mantenimiento

El funcionamiento consiste en el uso continuo y habitual de la aplicación por parte de los usuarios inmersos en un entorno productivo mientras ésta es útil.

Junto con el funcionamiento de la aplicación, se realizan tareas de mantenimiento para comprobar que está funcionando correctamente y adaptarla a cualquier nueva circunstancia que implique su modificación.

4. Errores.

Dependiendo del momento o fase en que se detecten, los errores se pueden clasificar en varias categorías.

Errores de compilación:

Los errores de compilación (o también conocidos como errores sintácticos) son causados por la ruptura de una regla sintáctica del lenguaje de programación, como una palabra reservada mal escrita, una oración incompleta, etc. Estos errores son los más fáciles de corregir, ya que son detectados por el compilador, que indicará dónde está el error y qué tipo de error se ha detectado a través de un mensaje de texto.

Errores de ejecución:

Por lo general, se deben a operaciones no permitidas, como dividir un número por cero, leer un valor no numérico cuando se esperaba un valor numérico, exceder un rango de valores permitidos, etc. Se detectan porque se produce una detención del programa mientras el programa se estaba ejecutando. Decimos que el sistema ha "abortado" el programa. Son más difíciles de detectar que los errores sintácticos porque pueden ocurrir o no dependiendo de los datos de entrada utilizados.

Errores lógicos:

Corresponden a la obtención de resultados erróneos por parte del programa. La única forma de detectarlos es diseñar una amplia gama de datos de prueba y comparar los datos resultantes obtenidos por el programa con los resultados correctos calculados por otros medios. Son los más difíciles de arreglar, no solo por la dificultad para detectarlos, sino porque son causados por el diseño del programa.

Errores de especificación:

Son el peor tipo de error y el más costoso de corregir. Son causados por especificaciones incorrectas debido a una mala comunicación por parte del programador y quien explica el problema (generalmente un cliente). Por lo general, se detectan cuando las aplicaciones ya se han instalado y puede implicar tener que volver a realizar una gran cantidad de trabajo.

5. La calidad de los programas

Para resolver un problema en particular, puede diseñar diferentes algoritmos o programas. La elección del más adecuado debe basarse en una serie de requisitos de calidad que tienen una gran importancia para valorar el coste de diseño y mantenimiento.

Las cualidades deseadas para un programa son:

Legibilidad:

Debe ser claro y fácil, de tal manera que sea fácil de leer y entender.

Portabilidad:

Su diseño debe permitir la codificación en diferentes lenguajes de programación y su instalación en diferentes sistemas.

Modificabilidad:

Debe facilitar su mantenimiento. Debe ser fácil de modificar y hacer las actualizaciones necesarias para adaptarlo a una nueva situación.

Eficacia:

Debe aprovechar al máximo los recursos informáticos, minimizando la memoria utilizada y el tiempo de ejecución o tiempo de proceso, siempre que no sea a expensas de los requisitos anteriores.



La metodología de programación es el conjunto de métodos y técnicas que ayudan a desarrollar programas que cumplan con todos los requisitos de calidad expuestos anteriormente.

6. Documentación de los programas

Para facilitar la operación y el mantenimiento de un programa es básico generar una documentación amplia, clara y precisa. Debe incluir las especificaciones del problema obtenidas en el análisis del problema y todos los detalles sobre cómo conseguir el máximo rendimiento o eficiencia.

Existen dos tipos de documentación en función de dónde sean internas y externas.

Documentación interna:

Está formado básicamente por el código fuente de los programas, y su objetivo principal es facilitar la lectura y comprensión de estos. Parte de esta documentación son:

Comentarios:

Son sentencias que se insertan en cualquier lugar del código fuente del programa y que son ignoradas por los compiladores o intérpretes. No se traducen a código objeto y, por lo tanto, no consumen memoria adicional. Debe incluir tantos comentarios como sea necesario para aclarar el significado de las líneas de código no obvias.

Código autocomentado:

Las palabras reservadas utilizadas en los lenguajes de programación constituyen por sí mismas parte de la documentación porque corresponden a términos (en inglés) que expresan su propósito.

Pero la documentación interna se puede mejorar mediante:

- Creación de identificadores adecuados y significativos para variables, constantes, funciones, etc...
- Declaración de constantes para valores fijos: Ej.: IVA = 0,21

- Sangría, paginación e inserción de líneas vacías para marcar bloques, bucles, etc...

Documentación externa:

Es el conjunto de documentos incluidos con los programas, pero sin formar parte de ellos.

- Especificaciones del análisis.
- Descripción y diagramas del diseño de los programas
- Manual de usuario
- Manual de mantenimiento